

Python OOP Interview Guide

Detailed OOP notes with Python examples and RAG project connections

Detailed interview preparation and practice notes. Text only. No tables or images.

Use this PDF as source data for your RAG chatbot and as revision material before interviews.

1. OOP introduction

Concept

Object-oriented programming organizes code using classes and objects. A class defines data and behavior, while an object is a real instance of the class.

Important details

OOP is useful for large projects because it groups related logic, improves reuse, and makes systems easier to extend.

Common mistakes

Do not use OOP just to look advanced. If a function is enough, use a function. Use OOP where state and behavior belong together.

Interview answer style

Explain OOP as a way to model real components and manage complexity.

Small code example

```
class Pipeline:  
    pass
```

Interview and practice focus

Be ready to explain oop introduction in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

2. Class and object

Concept

A class is a blueprint. An object is created from the class. Each object can have its own attributes.

Important details

Objects interact through methods. In Python, self refers to the current object and must be the first parameter of instance methods.

Common mistakes

Do not forget self in method definitions. Do not call instance attributes before setting them.

Interview answer style

Interview answer: class is a template, object is an instance with actual data.

Small code example

```
class Student:
    def __init__(self, name):
        self.name = name
```

Interview and practice focus

Be ready to explain class and object in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

3. Constructor `__init__`

Concept

`__init__` initializes an object after creation. It sets starting values for attributes.

Important details

The constructor does not create the object; `__new__` creates it. Usually you only customize `__init__`.

Common mistakes

Do not return a value from `__init__`.

Interview answer style

Say `__init__` is used to initialize object state.

Small code example

```
def __init__(self, data_dir="data"):
    self.data_dir = data_dir
```

Interview and practice focus

Be ready to explain constructor `__init__` in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

4. self keyword

Concept

self is a reference to the current object. It allows methods to access and modify object attributes.

Important details

Python passes self automatically when calling a method on an object. The name self is a convention, but it should always be used for readability.

Common mistakes

Do not pass self manually when calling obj.method().

Interview answer style

Explain self as the current object instance.

Small code example

```
obj.method() # Python passes obj as self
```

Interview and practice focus

Be ready to explain self keyword in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

5. Instance variables and class variables

Concept

Instance variables belong to individual objects. Class variables belong to the class and are shared across objects.

Important details

Use class variables for constants or shared configuration. Use instance variables for object-specific data.

Common mistakes

A common bug occurs when mutable class variables are shared accidentally across instances.

Interview answer style

Interview answer: instance data is per object, class data is shared.

Small code example

```
class User:
    role = "learner"
    def __init__(self, name): self.name = name
```

Interview and practice focus

Be ready to explain instance variables and class variables in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

6. Instance methods

Concept

Instance methods operate on object state and receive self. They are the most common kind of method.

Important details

Use methods to express behavior of an object. In a pipeline class, methods like `create_embedding` and `ask_question` are instance methods because they may use `self.vector_db`.

Common mistakes

Do not create methods that do unrelated tasks. Follow single responsibility.

Interview answer style

Explain instance methods as functions attached to objects.

Small code example

```
def ask_question(self, question):  
    return self.generator.ask(question)
```

Interview and practice focus

Be ready to explain instance methods in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

7. Encapsulation

Concept

Encapsulation keeps data and related methods together while hiding internal implementation details.

Important details

Python uses conventions rather than strict access modifiers. `_name` means internal use, `__name` triggers name mangling.

Common mistakes

Do not rely on name mangling as security. It is mainly to avoid accidental access or conflicts.

Interview answer style

Say encapsulation protects internal state and exposes clean public methods.

Small code example

```
class Store:
    def __init__(self):
        self._db = None
```

Interview and practice focus

Be ready to explain encapsulation in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

8. Properties

Concept

@property lets a method be accessed like an attribute. It is useful for computed values or validation.

Important details

A setter can validate values before assigning. This keeps object state valid.

Common mistakes

Do not overuse properties for expensive operations that look like simple attribute access.

Interview answer style

Interview answer: properties provide controlled access while keeping clean syntax.

Small code example

```
@property
def db(self):
    return self._db
```

Interview and practice focus

Be ready to explain properties in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

9. Inheritance

Concept

Inheritance lets a child class reuse and extend parent class behavior. It represents an is-a relationship.

Important details

Use inheritance for shared behavior, but prefer composition when the relationship is has-a.

Common mistakes

Do not create deep inheritance chains; they are hard to maintain.

Interview answer style

Interview answer: inheritance supports code reuse and specialization.

Small code example

```
class PdfLoader(DocumentLoader):  
    pass
```

Interview and practice focus

Be ready to explain inheritance in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

10. Method overriding

Concept

A child class overrides a parent method by defining a method with the same name.

Important details

This customizes behavior while keeping a common interface. Polymorphism often depends on overriding.

Common mistakes

Do not override methods without preserving expected behavior unless intentionally changing contract.

Interview answer style

Explain overriding using `Animal.speak` or `BaseLoader.load` examples.

Small code example

```
class Dog(Animal):  
    def speak(self): return "bark"
```

Interview and practice focus

Be ready to explain method overriding in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

11. super function

Concept

`super()` calls parent class methods. It is mostly used in child constructors and overridden methods.

Important details

It helps reuse parent initialization and prevents duplicate code.

Common mistakes

Do not hardcode parent class name unless necessary. `super()` is usually better.

Interview answer style

Interview answer: `super` gives access to parent behavior from child class.

Small code example

```
super().__init__(name)
```

Interview and practice focus

Be ready to explain `super` function in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

12. Polymorphism

Concept

Polymorphism means different objects can be used through the same interface.

Important details

If different loader classes all have `load()` method, the pipeline can call `loader.load()` without caring about the exact file type.

Common mistakes

Do not confuse method overloading with overriding. Python does not support traditional compile-time overloading like Java.

Interview answer style

Explain duck typing: if it behaves like needed, it can be used.

Small code example

```
def run(loader):  
    return loader.load()
```

Interview and practice focus

Be ready to explain polymorphism in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

13. Abstraction

Concept

Abstraction hides implementation details and exposes essential behavior. Abstract base classes can force child classes to implement required methods.

Important details

Use ABC when multiple classes must follow the same interface.

Common mistakes

Do not make everything abstract. Keep design simple.

Interview answer style

Interview answer: abstraction focuses on what an object does, not how it does it.

Small code example

```
from abc import ABC, abstractmethod
```

Interview and practice focus

Be ready to explain abstraction in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

14. Composition

Concept

Composition means one class contains objects of other classes. It models has-a relationships.

Important details

For a RAG pipeline, RagPipeLine has a DocumentLoader, Chunker, VectorStore, Retriever, and Generator. This is better than inheriting from all of them.

Common mistakes

Do not use inheritance when composition expresses the design better.

Interview answer style

Interview answer: composition is flexible and avoids tightly coupled inheritance hierarchies.

Small code example

```
self.loader = DocumentLoader()
```

Interview and practice focus

Be ready to explain composition in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

15. Static methods

Concept

@staticmethod creates a method that belongs to a class but does not use self or cls.

Important details

Use it for utility behavior related to the class concept but not dependent on instance state.

Common mistakes

Do not use staticmethod when an ordinary module-level function is more appropriate.

Interview answer style

Say static methods are namespaced utility methods.

Small code example

```
@staticmethod
def clean(text): return text.strip()
```

Interview and practice focus

Be ready to explain static methods in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

16. Class methods

Concept

@classmethod receives cls and can access class variables or create alternate constructors.

Important details

They are useful for factory methods such as from_config or from_env.

Common mistakes

Do not use classmethod when you need instance state; use instance methods.

Interview answer style

Interview answer: class methods work with the class rather than a specific object.

Small code example

```
@classmethod
def from_env(cls): return cls(os.getenv("PATH"))
```

Interview and practice focus

Be ready to explain class methods in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

17. Dunder methods

Concept

Dunder methods customize built-in behavior. Examples include `__str__`, `__repr__`, `__len__`, `__eq__`, `__call__`, and `__enter__`.

Important details

They make custom objects integrate naturally with Python syntax.

Common mistakes

Do not implement dunder methods incorrectly because built-in behavior may become confusing.

Interview answer style

Explain `__str__` for user-friendly string and `__repr__` for developer-friendly representation.

Small code example

```
def __str__(self):  
    return self.name
```

Interview and practice focus

Be ready to explain dunder methods in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

18. Dataclasses

Concept

dataclass automatically creates init, repr, and comparison methods for data-focused classes.

Important details

Use dataclasses for simple containers like configuration, request metadata, or settings.

Common mistakes

Do not use dataclass for classes with heavy behavior and complex invariants unless it still fits.

Interview answer style

Interview answer: dataclasses reduce boilerplate for classes that mainly store data.

Small code example

```
from dataclasses import dataclass
@dataclass
class Config:
    top_k: int = 3
```

Interview and practice focus

Be ready to explain dataclasses in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

19. OOP design principles

Concept

Important principles include single responsibility, open-closed, dependency inversion, low coupling, and high cohesion.

Important details

A class should have one main reason to change. Pipeline classes should coordinate components, not perform every low-level task.

Common mistakes

Do not create god classes that load files, clean text, create embeddings, call APIs, and format UI all in one place.

Interview answer style

Explain with your RAG project module separation.

Small code example

```
class RagPipeLine:
    def create_embedding(self): pass
```

Interview and practice focus

Be ready to explain oop design principles in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

20. OOP in RAG project

Concept

A RAG project becomes cleaner when each class handles one part: DocumentLoader loads files, TextCleaner cleans text, Chunker splits text, Embeddings returns model, VectorStore stores vectors, Retriever gets context, Generation creates answer.

Important details

This design is interview-friendly because you can explain responsibilities clearly.

Common mistakes

Do not recreate vector DB in ask_question for every query. Store it in self.vector_db after loading or creating it.

Interview answer style

Strong answer: OOP helps organize the RAG pipeline into reusable, testable components.

Small code example

```
self.vector_db = vectorize.load_vector_store(model)
```

Interview and practice focus

Be ready to explain oop in rag project in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

21. OOP interview practice questions

Concept

Practice questions include: What is OOP? What are pillars of OOP? Difference between class and object? What is self? Inheritance vs composition? Polymorphism example? What is abstraction?

Important details

Practice coding a small class with constructor, method, property, inheritance, and composition.

Common mistakes

Do not memorize only definitions; explain with a project example.

Interview answer style

Use your RAG pipeline as the main example in interviews.

Small code example

```
print("Practice OOP with small classes")
```

Interview and practice focus

Be ready to explain oop interview practice questions in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.